LAPORAN PRAKTIKUM PTM 9

Autentikasi dan Manajemen Sesi Menggunakan Supabase pada Aplikasi To-Do List Berbasis Flutter

Penerapan Supabase Auth, Row Level Security, dan penyimpanan sesi untuk menjamin setiap pengguna hanya mengakses data miliknya sendiri.

Disusun oleh:

RIZAL SURYAWAN

NIM 241011750067

Kelas 04SIFE002

6 November 2026

1. Pendahuluan

1.1 Latar Belakang

Aplikasi pencatat tugas (To-Do List) menyimpan data yang bersifat pribadi bagi setiap penggunanya. Tanpa mekanisme autentikasi, seluruh data akan terlihat oleh siapa pun yang membuka aplikasi, dan tanpa pemisahan data di sisi server, seorang pengguna dapat membaca atau mengubah data milik pengguna lain. Oleh karena itu, aplikasi To-Do List membutuhkan dua hal sekaligus: autentikasi untuk memastikan identitas pengguna, dan otorisasi untuk membatasi akses terhadap data.

Pada praktikum ini, autentikasi dan manajemen sesi dibangun menggunakan Supabase. Supabase menyediakan layanan autentikasi siap pakai (Supabase Auth) dengan mekanisme email dan password, serta basis data PostgreSQL yang dilengkapi fitur *Row Level Security* (RLS). Dengan RLS, aturan pembatasan akses data ditetapkan langsung pada basis data sehingga isolasi data antar pengguna tidak bergantung pada kode di sisi aplikasi. Aplikasi klien dikembangkan menggunakan Flutter dengan paket `supabase_flutter`.

1.2 Tujuan

1. Membuat fitur login menggunakan email dan password pada aplikasi Flutter.
1. Menampilkan email pengguna yang sedang login pada halaman utama aplikasi.
1. Membuat sesi login tetap aktif ketika aplikasi ditutup lalu dibuka kembali.
1. Menyediakan tombol logout untuk keluar dari aplikasi.
1. Memastikan setiap akun hanya dapat melihat data To-Do List miliknya sendiri, dibuktikan melalui pengujian dua akun berbeda.

1.3 Lingkungan Pengembangan

Komponen	Keterangan
Framework aplikasi	Flutter 3.44.4 (Dart 3.12.2), channel stable
Paket autentikasi	supabase_flutter 2.18.3
Backend	Supabase (PostgreSQL, Supabase Auth, Row Level Security)
Perangkat uji	Emulator Android 17 (API 37), arsitektur x86_64
Alat uji	flutter analyze, flutter test, integration_test
Bahasa antarmuka	Bahasa Indonesia

2. Dasar Teori

2.1 Supabase dan Supabase Auth

Supabase adalah layanan *Backend as a Service* yang dibangun di atas PostgreSQL. Supabase Auth mengelola data pengguna pada skema `auth` dan menyediakan endpoint registrasi (`signUp`) serta login (`signInWithPassword`). Setelah login berhasil, server mengembalikan *access token* dan *refresh token* dalam bentuk JSON Web Token (JWT) yang memuat identitas pengguna pada klaim `sub`.

2.2 Sesi dan Token

Access token berumur pendek dan dipakai pada setiap permintaan data, sedangkan refresh token berumur panjang dan dipakai untuk memperbarui access token. Paket `supabase_flutter` menyimpan kedua token tersebut pada penyimpanan lokal perangkat secara otomatis (opsi `persistSession` aktif secara bawaan). Karena token tersimpan di perangkat, pengguna tidak perlu login ulang ketika aplikasi ditutup dan dibuka kembali.

2.3 Row Level Security (RLS)

RLS adalah fitur PostgreSQL yang memungkinkan penulisan kebijakan (*policy*) pada setiap tabel. Kebijakan tersebut dievaluasi oleh server untuk setiap baris data yang diakses. Supabase menyediakan fungsi `auth.uid()` yang mengembalikan id pengguna dari JWT yang sedang dipakai, sehingga kebijakan dapat ditulis sebagai `auth.uid() = user_id`. Dengan cara ini, permintaan untuk membaca atau mengubah data milik pengguna lain akan ditolak oleh basis data, meskipun permintaan tersebut dibuat langsung ke REST API tanpa melalui aplikasi.

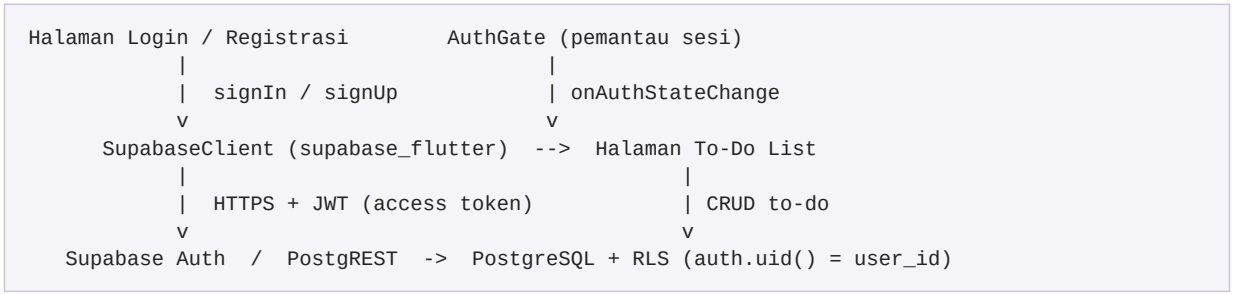
2.4 Isolasi Data di Sisi Server

Kunci API yang dipakai aplikasi klien (publishable key) memang dirancang untuk dapat disebar bersama aplikasi. Karena itu, keamanan data tidak boleh bergantung pada kerahasiaan kunci tersebut, melainkan pada kebijakan RLS yang berjalan di server. Inilah alasan utama RLS wajib diaktifkan pada tabel yang menyimpan data pengguna.

3. Perancangan Sistem

3.1 Arsitektur Aplikasi

Aplikasi dibagi menjadi empat lapisan. Lapisan tampilan terdiri dari halaman login, halaman registrasi, dan halaman To-Do. Lapisan pengendali sesi adalah `AuthGate` yang memilih halaman berdasarkan ada atau tidaknya sesi aktif. Lapisan akses data adalah `SupabaseClient` dari paket `supabase_flutter` yang melakukan permintaan ke Supabase Auth dan PostgREST. Lapisan terakhir adalah basis data PostgreSQL yang dilindungi RLS.



3.2 Alur Autentikasi

1. Pengguna mengisi email dan password pada halaman registrasi. Aplikasi memanggil `signUp()`, lalu Supabase membuat baris pengguna baru pada tabel `auth.users`.
1. Pengguna mengisi email dan password pada halaman login. Aplikasi memanggil `signInWithPassword()` dan menerima sesi berisi access token dan refresh token.
1. `AuthGate` mendengarkan aliran `onAuthStateChange`. Saat sesi muncul, halaman To-Do ditampilkan; saat sesi hilang, halaman login ditampilkan kembali.
1. Setiap operasi data dikirim bersama access token. PostgreSQL mengevaluasi kebijakan RLS berdasarkan `auth.uid()` sehingga hanya baris milik pengguna yang bersangkutan yang dapat diakses.
1. Tombol logout memanggil `signOut()` yang menghapus sesi dari perangkat sehingga pengguna kembali ke halaman login.

3.3 Struktur Tabel To-Do

Kolom	Tipe data	Keterangan
`id`	`uuid`	Primary key, nilai bawaan gen_random_uuid()
`user_id`	`uuid`	Pemilik data; nilai bawaan auth.uid(), mengacu ke auth.users
`title`	`text`	Judul to-do, dibatasi 1 sampai 200 karakter
`is_done`	`boolean`	Status selesai, nilai bawaan false
`created_at`	`timestamptz`	Waktu pembuatan, nilai bawaan now()

3.4 Kebijakan Row Level Security

Tabel `todos` dikunci dengan empat kebijakan berikut. Semuanya hanya berlaku untuk peran `authenticated`, yaitu pengguna yang sudah login.

Operasi	Nama policy	Aturan
SELECT	`todos_select_own`	`using (auth.uid() = user_id)`
INSERT	`todos_insert_own`	`with check (auth.uid() = user_id)`
UPDATE	`todos_update_own`	`using dan with check (auth.uid() = user_id)`
DELETE	`todos_delete_own`	`using (auth.uid() = user_id)`

4. Implementasi

4.1 Struktur Direktori

```
lib/
  main.dart                 inisialisasi Supabase + MaterialApp
  config/supabase_config.dart   kredensial dari --dart-define
  models/todo.dart          model data To-Do
  screens/auth_gate.dart    pemantau sesi (login atau To-Do)
  screens/login_screen.dart     form login
  screens/register_screen.dart  form registrasi
  screens/todo_screen.dart      daftar to-do, email pengguna, tombol logout
  screens/setup_required_screen.dart  petunjuk bila kredensial belum diisi
  utils/validators.dart     validasi email dan password
  utils/error_messages.dart terjemahan pesan error Supabase
supabase/schema.sql         tabel todos + kebijakan RLS
test/widget_test.dart       unit test dan widget test
integration_test/account_isolation_test.dart  uji isolasi dua akun
```

4.2 Konfigurasi Kredensial

Kredensial Supabase tidak ditulis langsung di dalam kode, melainkan dibaca melalui `--dart-define` pada berkas `dart_defines.json`. Bila kredensial belum diisi, aplikasi menampilkan halaman petunjuk konfigurasi alih-alih pesan kesalahan yang membingungkan.

```
class SupabaseConfig {
  static const String url = String.fromEnvironment('SUPABASE_URL');
  static const String publishableKey =
      String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');

  static bool get isConfigured => url.isNotEmpty && publishableKey.isNotEmpty;
}
```

4.3 Inisialisasi dan Pemantauan Sesi

Fungsi `main()` melakukan inisialisasi Supabase sebelum antarmuka dijalankan. Selanjutnya `AuthGate` memantau perubahan status autentikasi dan menentukan halaman yang ditampilkan. Karena sesi dipulihkan dari penyimpanan lokal saat aplikasi dimulai, nilai awal `currentSession` langsung dipakai sebagai data awal sehingga pengguna tidak melihat kedipan halaman login.

```
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  if (SupabaseConfig.isConfigured) {
    await Supabase.initialize(
      url: SupabaseConfig.url,
      publishableKey: SupabaseConfig.publishableKey,
    );
  }

  runApp(const TodoApp());
}
```

```
final auth = Supabase.instance.client.auth;

return StreamBuilder<AuthState>(
  stream: auth.onAuthStateChange,
  initialData: AuthState(AuthChangeEvent.initialSession, auth.currentSession),
  builder: (context, snapshot) {
    final session = snapshot.data?.session ?? auth.currentSession;

    if (session == null) {
      return const LoginScreen();
    }
    return const TodoScreen();
  },
);
```

4.4 Halaman Login dan Registrasi

Halaman login memakai komponen `Form` dengan validasi email dan password minimal enam karakter. Kesalahan yang dikembalikan Supabase diterjemahkan menjadi pesan berbahasa Indonesia, misalnya “Email atau password salah” untuk kesalahan `invalid login credentials`.

```
Future<void> _submit() async {
  if (!_formKey.currentState!.validate()) {
    return;
  }

  setState(() {
    _loading = true;
    _error = null;
  });

  try {
    await Supabase.instance.client.auth.signInWithPassword(
      email: _emailController.text.trim(),
      password: _passwordController.text,
    );
  } catch (error) {
    if (!mounted) {
      return;
    }
    setState(() => _error = messageForError(error));
  } finally {
    if (mounted) {
      setState(() => _loading = false);
    }
  }
}
```

> Catatan: jika login berhasil, AuthGate otomatis berpindah ke halaman To-Do melalui aliran onAuthStateChange, sehingga halaman login tidak perlu melakukan navigasi manual.

Halaman registrasi memanggil `signUp()` dan menangani dua kemungkinan. Bila proyek Supabase mewajibkan konfirmasi email, server tidak mengembalikan sesi; aplikasi menampilkan dialog agar pengguna memeriksa kotak masuk emailnya. Bila konfirmasi email dimatikan, sesi langsung diterima dan pengguna masuk otomatis.

4.5 Menampilkan Email Pengguna dan Tombol Logout

Email pengguna diambil dari objek sesi yang sedang aktif lalu ditampilkan pada kartu akun di bagian atas halaman To-Do. Kartu yang sama memuat tombol Logout dengan dialog konfirmasi agar pengguna tidak keluar secara tidak sengaja.

```
String get _email => _client.auth.currentUser?.email ?? '(email tidak tersedia)';

Future<void> _logout() async {
  // menampilkan dialog konfirmasi terlebih dahulu
  final confirmed = await showDialog<bool>( /* ... */ );
  if (confirmed != true) {
    return;
  }

  await _client.auth.signOut();
}
```

4.6 Operasi Data To-Do

Seluruh operasi data dilakukan melalui PostgREST. Penulisannya sengaja sederhana karena keamanan sudah ditangani RLS, bukan oleh filter di sisi aplikasi.

```
final rows = await _client
    .from('todos')
    .select()
    .order('is_done', ascending: true)
    .order('created_at', ascending: false);

await _client.from('todos').insert({
  'title': title,
  'user_id': _client.auth.currentUser!.id,
});

await _client.from('todos').update({'is_done': !todo.isDone}).eq('id', todo.id);
await _client.from('todos').delete().eq('id', todo.id);
```

4.7 Skema dan Kebijakan pada Supabase

Berkas `supabase/schema.sql` dijalankan satu kali pada SQL Editor Supabase. Berkas tersebut membuat tabel serta mengaktifkan RLS beserta seluruh kebijakannya.

```
create table if not exists public.todos (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  title text not null check (char_length(btrim(title)) between 1 and 200),
  is_done boolean not null default false,
  created_at timestamptz not null default now()
);

alter table public.todos enable row level security;

create policy "todos_select_own" on public.todos
  for select to authenticated using (auth.uid() = user_id);

create policy "todos_insert_own" on public.todos
  for insert to authenticated with check (auth.uid() = user_id);

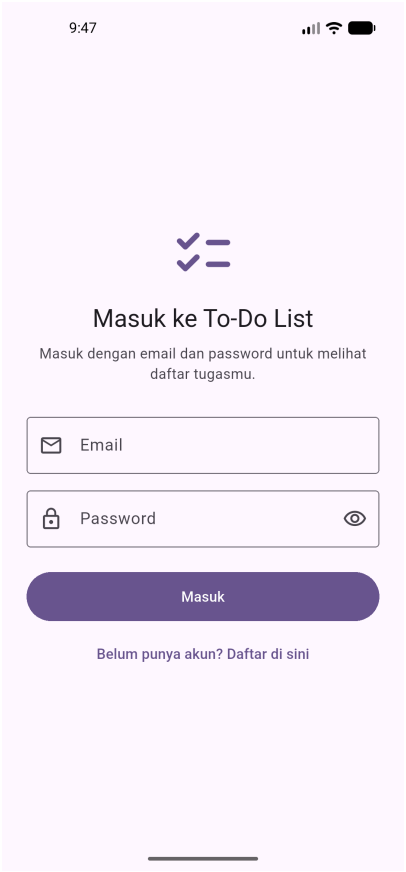
create policy "todos_update_own" on public.todos
  for update to authenticated
  using (auth.uid() = user_id) with check (auth.uid() = user_id);

create policy "todos_delete_own" on public.todos
  for delete to authenticated using (auth.uid() = user_id);
```

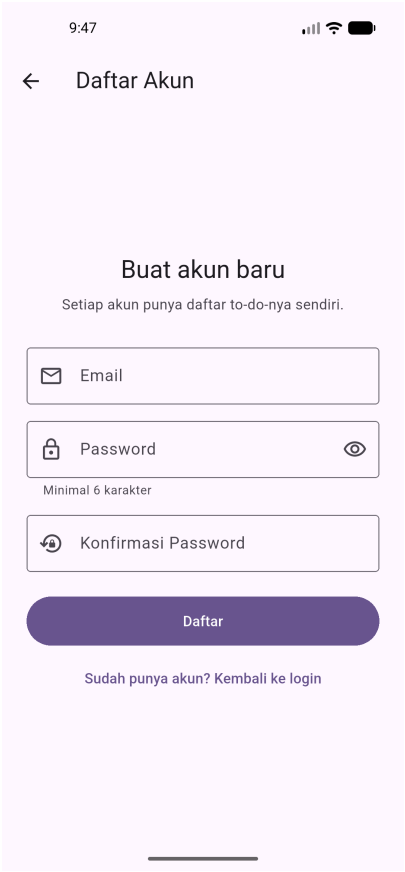
5. Hasil

Bagian ini menampilkan tangkapan layar aplikasi yang dijalankan pada emulator Android. Seluruh fitur yang diminta pada praktikum ini sudah berjalan.

5.1 Halaman Login dan Registrasi

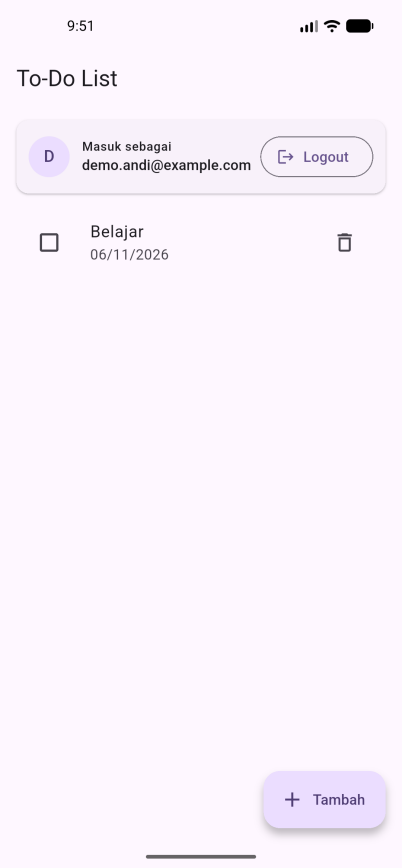


Gambar 5.1 Halaman login dengan email dan password

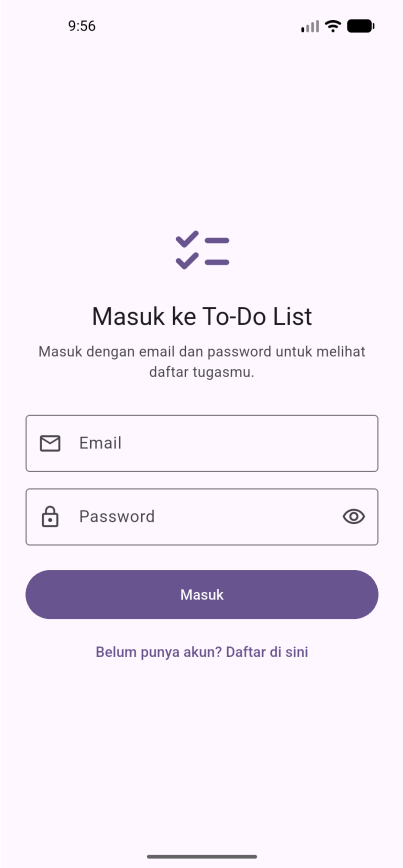


Gambar 5.2 Halaman registrasi akun baru

5.2 Halaman To-Do List dan Identitas Pengguna



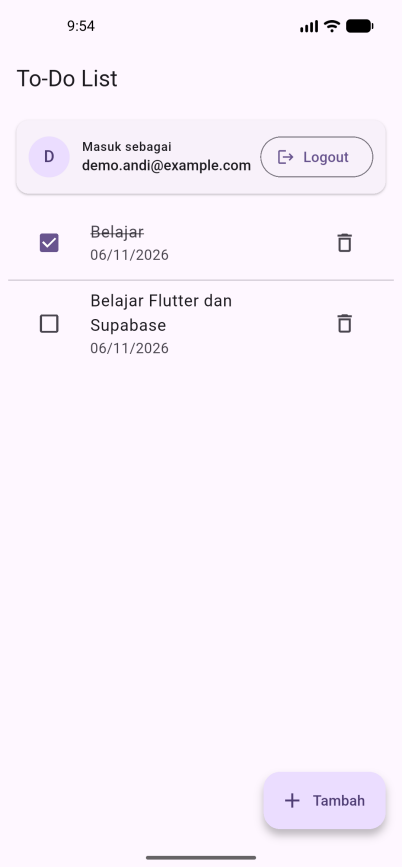
Gambar 5.3 Kartu akun menampilkan email pengguna, tombol Logout, dan daftar to-do miliknya



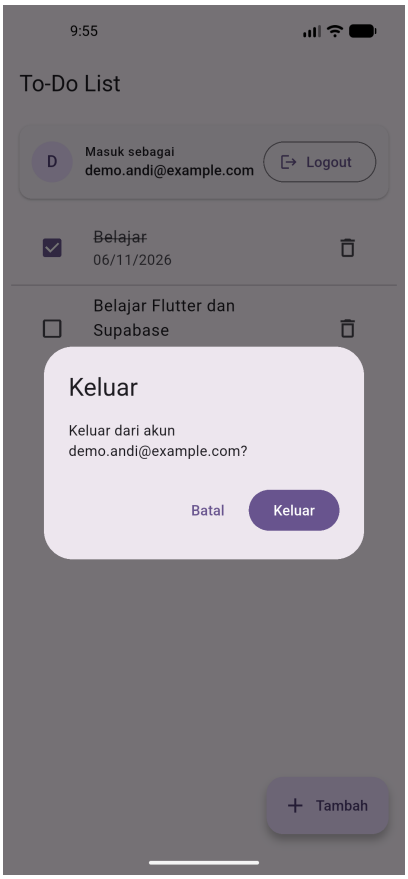
Gambar 5.4 Tampilan setelah logout, kembali ke halaman login

5.3 Sesi Tetap Aktif Setelah Aplikasi Ditutup

Pengujian dilakukan dengan menghentikan paksa proses aplikasi melalui perintah `adb shell am force-stop`, kemudian membukanya kembali. Aplikasi langsung menampilkan halaman To-Do dengan email pengguna tanpa meminta login ulang, yang membuktikan sesi tersimpan dengan baik di perangkat.



Gambar 5.5 Setelah aplikasi ditutup paksa dan dibuka kembali, pengguna tetap login sebagai demo.andi@example.com



Gambar 5.6 Dialog konfirmasi sebelum logout

6. Pengujian

6.1 Pengujian Otomatis

Tiga tingkat pengujian dijalankan. Analisis statis memastikan tidak ada kesalahan maupun peringatan pada kode. Unit test dan widget test menguji validasi masukan, model data, serta perilaku halaman login tanpa memerlukan jaringan. Uji integrasi berjalan pada emulator dan terhubung ke proyek Supabase sungguhan untuk memverifikasi isolasi data dua akun.

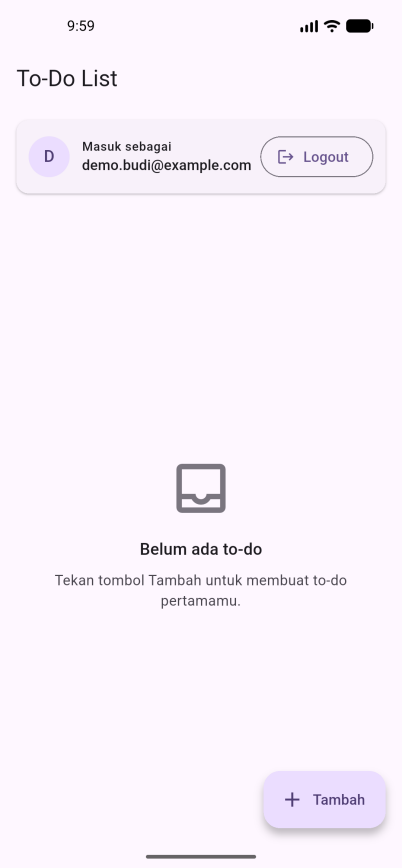
Pengujian	Perintah	Hasil
Analisis statis	`flutter analyze`	No issues found
Unit dan widget test	`flutter test`	9 pengujian lulus
Uji isolasi dua akun	`flutter test integration_test`	1 pengujian lulus

6.2 Pengujian Manual dengan Dua Akun

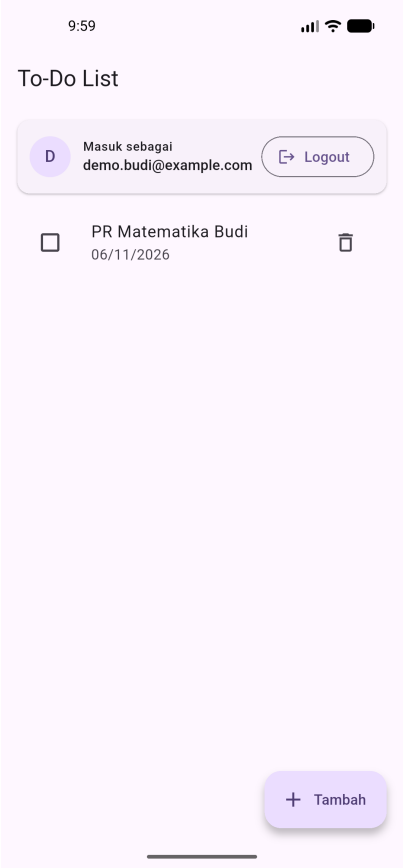
Pengujian manual dilakukan dengan membuat dua akun berbeda pada aplikasi, yaitu `demo.andi@example.com` dan `demo.budi@example.com`. Setiap akun diberi data to-do, kemudian diperiksa data mana yang tampil pada masing-masing akun.

No	Skenario	Hasil yang diharapkan	Hasil
1	Registrasi akun A melalui halaman registrasi	Registrasi berhasil dan langsung masuk ke halaman To-Do	Sesuai
2	Menambah dua to-do pada akun A	Kedua to-do muncul pada daftar akun A	Sesuai
3	Menutup paksa lalu membuka kembali aplikasi	Masih login sebagai akun A tanpa mengisi password	Sesuai
4	Menekan tombol Logout	Sesi berakhir dan kembali ke halaman login	Sesuai
5	Registrasi akun B, membuka halaman To-Do	Daftar kosong; dua to-do akun A tidak terlihat	Sesuai
6	Menambah satu to-do pada akun B	Hanya satu to-do milik B yang tampil	Sesuai
7	Logout lalu login kembali sebagai akun A	Hanya dua to-do milik A yang tampil	Sesuai

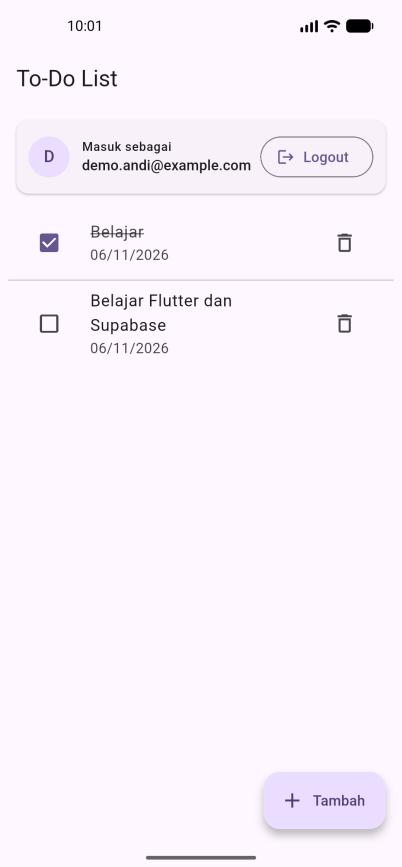
6.3 Bukti Isolasi Data



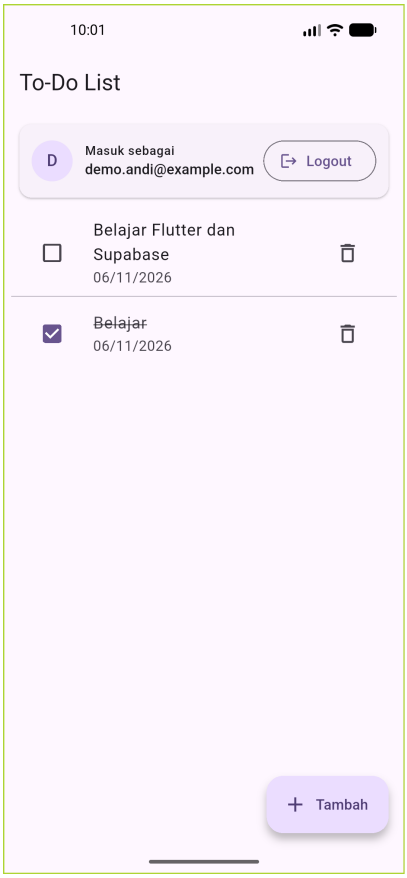
Gambar 6.1 Akun demo.budi@example.com melihat daftar kosong, data akun demo.andi tidak terlihat



Gambar 6.2 Setelah menambah data, akun B hanya melihat satu to-do miliknya sendiri



Gambar 6.3 Login kembali sebagai akun A: hanya dua to-do milik A yang tampil, to-do milik B tidak ada



Gambar 6.4 Urutan daftar setelah perbaikan, to-do yang belum selesai tampil di atas

6.4 Hasil Uji Integrasi Isolasi Data

Uji integrasi menjalankan skenario berikut secara otomatis dengan memakai dua akun uji yang dibuat pada saat pengujian berjalan.

Skenario pengujian	Hasil
Akun A membuat dua to-do, lalu membaca seluruh baris tabel	Tepat dua baris, semuanya milik A
Akun B membaca seluruh baris tabel setelah login	Hanya satu baris, yaitu milik B
Akun B mencoba membaca to-do milik A dengan menebak id-nya	Hasil kosong, data tidak terbaca
Akun B mencoba menambah to-do dengan user_id milik A	Ditolak server dengan PostgrestException
Akun B mencoba mengubah dan menghapus to-do milik A	Tidak berpengaruh; data A tetap utuh saat diperiksa kembali

6.5 Temuan Selama Pengembangan

1. **Urutan daftar tidak sesuai harapan.** Fungsi `order()` pada pustaka postgrest memiliki nilai bawaan `ascending: false`, sehingga to-do yang sudah selesai justru tampil di urutan teratas. Perbaikan dilakukan dengan menuliskan parameter `ascending: true` secara eksplisit agar to-do yang belum selesai tampil lebih dahulu.
1. **Konfirmasi email memengaruhi alur registrasi.** Ketika pengaturan konfirmasi email aktif, `signUp()` tidak mengembalikan sesi sehingga pengguna belum dapat langsung login. Aplikasi menangani kedua kemungkinan ini dan menampilkan petunjuk yang sesuai kepada pengguna.
1. **Validasi di sisi aplikasi tidak cukup.** Menyaring data berdasarkan id pengguna di kode Flutter tidak menjamin keamanan karena permintaan dapat dikirim langsung ke REST API. Pembatasan pada server melalui RLS terbukti menolak percobaan penulisan data atas nama pengguna lain.

7. Kesimpulan

1. Fitur login dengan email dan password berhasil dibuat menggunakan Supabase Auth melalui fungsi `signUp()` dan `signInWithPassword()`, lengkap dengan validasi masukan dan pesan kesalahan berbahasa Indonesia.
1. Email pengguna yang sedang login ditampilkan pada kartu akun di halaman To-Do, diambil dari objek sesi aktif.
1. Sesi login tetap aktif setelah aplikasi ditutup dan dibuka kembali karena token disimpan otomatis di perangkat; hal ini dibuktikan melalui pengujian dengan menghentikan paksa proses aplikasi.
1. Tombol Logout berfungsi mengakhiri sesi dan mengembalikan aplikasi ke halaman login.
1. Isolasi data antar pengguna terjamin oleh kebijakan Row Level Security pada tabel `todos`. Pengujian dengan dua akun menunjukkan bahwa setiap akun hanya dapat melihat, menambah, mengubah, dan menghapus to-do miliknya sendiri.
1. Keamanan data tidak bergantung pada kerahasiaan kunci API, melainkan pada kebijakan yang dijalankan oleh basis data di sisi server.

Lampiran

A. Cara Menjalankan Aplikasi

```
# 1. Isi dart_defines.json dengan URL dan publishable key Supabase
# 2. Jalankan skema tabel dan kebijakan RLS (supabase/schema.sql) pada SQL Editor
# 3. Jalankan aplikasi
flutter run --dart-define-from-file=dart_defines.json
```

B. Akun Uji

Email	Keterangan
`demo.andi@example.com`	Akun A, memiliki dua to-do
`demo.budi@example.com`	Akun B, memiliki satu to-do

C. Repositori

Kode sumber tersedia pada repositori GitHub `zalwan/flutter_application_1` dengan riwayat commit terpisah per topik pada branch `praktikum-ptm-9`.